SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

Day 2: The Software Development Life Cycle

Understanding how an idea becomes a dependable software product—from planning and requirements to release and maintenance.

THE BIG QUESTION
If a college wants an app for event registration, what work must happen before the app is ready for real students to use—and what must happen after launch?

In this chapter

- Understand the purpose of the SDLC and why teams use it.
- Explore each SDLC phase through one continuous real-world example.
- Identify typical activities, documents, decisions, and deliverables in every phase.
- Understand feedback, iteration, and change across the lifecycle.
- Practise applying the SDLC to a College Event Management System.

A student-facing chapter with explanations, examples, diagrams, activities, review questions, suggested answers, and homework.

1. Why Do We Need a Life Cycle?

In Day 1, we discussed that software engineering is more than writing code. A software product begins with a problem, not with a programming language. Teams need to understand who has the problem, what outcome is expected, how the solution should work, how quality will be checked, and how the product will be supported after release.

Imagine that your college asks a developer to “make an event app.” If the developer starts coding immediately, important questions may be missed. Can students register for multiple events? Is there a maximum capacity? Can registrations be cancelled? Who creates events? Can faculty approve an event? What happens if two students try to take the final seat at the same time? How will attendance be recorded?

The Software Development Life Cycle (SDLC) provides a way to organise these activities so the team can make decisions in a sensible order, check its work, and respond to new information.

A life cycle is not just a list of phases

The word *cycle* is important. Software is rarely created perfectly in one attempt. Testing may reveal a design problem. Users may discover that a requirement was misunderstood. A security review may require a change. A product released today may need updates next month. The team therefore revisits decisions as it learns.

> **CORE IDEA**
> The SDLC is a structured way to plan, develop, test, deploy, operate, and maintain software. It helps teams manage work, quality, time, cost, risks, and changing needs.

The running example: College Event Management System (CEMS)

Throughout this chapter, imagine a college that organises technical festivals, cultural programmes, workshops, and guest lectures. Today, students register using different forms and spreadsheets. Organisers manually remove duplicate entries, count registrations, send confirmations, and prepare attendance lists. The college wants one portal to manage these activities.

The proposed system should let students browse events, register, cancel when allowed, and receive confirmation. Organisers should create events, set capacities, view registrations, and download reports. Administrators should manage access and ensure that data is handled appropriately.

The CEMS example gives us a consistent way to see how each SDLC phase contributes to the final product.

2. The SDLC at a Glance

A COMMON SDLC VIEW

IDEA → PLANNING → REQUIREMENTS → DESIGN → IMPLEMENTATION → TESTING → DEPLOYMENT → MAINTENANCE

Feedback, defects, and new needs may take the team back to an earlier phase. Different organisations may combine or name phases differently.

What does each phase contribute?

Phase	Main question	Typical output
Planning	Why build it, and is it feasible?	Goals, scope, feasibility, risks, initial schedule
Requirements	What should the system do?	Requirements, use cases or user stories, acceptance criteria
Design	How will the system be organised?	Architecture, diagrams, screen designs, data model
Implementation	How will we build it?	Source code and working components
Testing	Does it meet expectations?	Test cases, defect reports, test results
Deployment	How do users access it?	Release, configuration, rollout and recovery plan
Maintenance	How will it remain useful?	Fixes, updates, monitoring, support and enhancements

Do all projects follow the exact same sequence?

Not necessarily. A traditional project may complete much of one phase before moving to the next. An iterative team may repeat requirements, design, implementation, and testing in short cycles. A small project may combine some activities. A safety-critical or highly regulated project may require formal reviews and evidence at particular stages. The SDLC gives us the activities to manage; a process model helps determine how to organise them.

Think of planning a college festival

Planning a festival offers a useful analogy. Before the event, organisers define the purpose, budget, venue, participants, schedule, safety arrangements, and responsibilities. They then arrange the venue and services, check readiness, open the event, and handle issues. Afterward, they collect feedback and improve the next festival. Software development also needs preparation, execution, checks, launch, and ongoing improvement.

IMPORTANT DISTINCTION

The SDLC is the broad lifecycle of software work. Waterfall, Incremental, Spiral, Prototype-based development, and Agile are different ways to organise that work. These approaches will be explored in more detail in the following classes.

3. Phase One — Planning

Planning answers a fundamental question: should we build this system, and if so, what should the first version achieve? A team identifies the problem, intended users, expected benefits, boundaries, constraints, resources, and major risks.

For CEMS, the team might agree that the first release should support event listings, student registration, cancellation rules, and basic organiser reports. QR attendance, certificates, payments, and feedback surveys may be considered for later releases. Defining what is not included is just as important as defining what is included.

Key planning questions

- What problem are we solving, and who experiences it?
- Who are the stakeholders and decision-makers?
- What features are essential for the first release?
- What is the deadline, and are there fixed dates such as the annual festival?
- What people, skills, technology, budget, and infrastructure are available?
- What risks could delay or damage the project?
- How will we decide whether the project has succeeded?

Feasibility: Is the idea practical?

A feasibility study considers whether the proposed solution is realistic. The details vary by organisation, but common areas include technical feasibility, economic feasibility, operational feasibility, schedule feasibility, and legal or compliance constraints.

Feasibility area	Question for CEMS
Technical	Can the college support the required hosting, authentication, database, and expected user load?
Economic	Are the development, hosting, support, and maintenance costs justified by the benefits?
Operational	Will students and event organisers be able to use the system? Who will support it?
Schedule	Can a useful and sufficiently tested version be ready before the festival?
Legal / policy	How should student information be collected, accessed, retained, and protected?

Scope and success criteria

Scope defines the boundaries of the project. A statement such as “build an event portal” is too broad to guide work. A more useful initial scope could state: “The first release will allow authenticated students to view published events and register, while authorised organisers can create events and download registration lists.”

Success criteria make the outcome observable. For example, the team may agree to support a specified number of concurrent users, prevent duplicate registrations, and provide organisers with accurate registration reports. Numeric performance targets should be agreed using realistic needs and test conditions.

Planning risks

A risk is an uncertain event that could affect the project. CEMS risks might include a deadline that cannot be met, unavailable college login integration, unexpected traffic on registration day, or unclear ownership of student data. Recording a risk does not make it disappear, but it helps the team plan a response.

CEMS PLANNING OUTPUT

> A short project brief containing the problem, stakeholders, initial scope, constraints, success criteria, feasibility findings, major risks, and a preliminary schedule.

4. Phase Two — Requirements

Requirements explain what users need the system to do and what conditions it must satisfy. This phase is about understanding the problem accurately—not guessing what users mean and not jumping straight to screens or code.

Imagine asking a student, “What do you need from the event portal?” The student might say, “I want to register easily.” That statement expresses a need, but the team must ask follow-up questions. Should the student sign in first? Can the student register for two overlapping events? What happens if the event is full? Should a confirmation be sent by email, SMS, or shown on screen?

Functional and non-functional requirements

Functional requirements describe services, actions, or behaviour the system must provide. **Non-functional requirements** describe quality attributes or constraints such as security, performance, availability, accessibility, or compatibility.

Type	CEMS example	How it can be checked
Functional	A student can register for a published event that has available capacity.	Register a valid student for an open event and verify the registration record.
Functional	The system prevents duplicate registration for the same event.	Attempt to register the same student twice and check the result.
Functional	An authorised organiser can download a registration report.	Sign in as an organiser, download the report, and verify its contents.
Non-functional	Only authorised organisers can access private registration details.	Attempt access as a student and confirm that access is denied.
Non-functional	The event list meets an agreed response-time target under a defined load.	Run a performance test using the agreed workload and measure response times.
Non-functional	The portal remains usable on supported mobile and desktop browsers.	Test the main flows on the agreed devices and browsers.

Why vague requirements are risky

“The website must be fast,” “the app should be secure,” and “registration should work properly” are difficult to test because they do not define clear conditions. A better requirement states the expected behaviour or measurable target, while leaving room for the team to choose a suitable technical solution.

Elicitation: learning from stakeholders

Requirements elicitation is the process of discovering needs and constraints. Teams may interview students and organisers, observe current registration work, run workshops, study existing forms, examine support complaints, or create prototypes. Different methods reveal different kinds of information: an interview may expose a policy, while observation may reveal an inefficient manual step.

Example questions for a student

- How do you currently discover events?
- What information do you need before deciding to register?
- What should happen if an event is full?
- Can you cancel a registration? If yes, until when?
- How should you know that registration succeeded?
- What personal information should not be visible to other students?

Requirements validation

Validation checks that requirements describe the right system and are clear, consistent, feasible, and testable. A stakeholder review may uncover conflicting rules. A prototype may reveal that users misunderstand a screen. A testable requirement helps both the developers and testers understand what “done” means.

> **CEMS REQUIREMENTS OUTPUT**
> A reviewed requirements list, functional and non-functional requirements, user roles, important business rules, acceptance criteria, assumptions, and unresolved questions.

5. Phase Three — Design

Design answers the question: how will the system meet its requirements? It translates the needs into a technical and user-facing plan. Design decisions may cover system architecture, components, data storage, user-interface layouts, API contracts, security, error handling, and deployment environments.

A design is not merely a colourful screen. A good screen can still be connected to an unreliable data model or an insecure service. Design considers how the whole system behaves, including what happens when things go wrong.

CEMS design decisions

- Separate the student experience from organiser administration.
- Define records for students, events, and registrations.
- Decide how the system verifies a student's identity and role.
- Specify how capacity is enforced when many students register at once.
- Define what confirmation means and how retries avoid duplicate records.
- Decide how reports can be accessed and exported safely.

A simple architecture

Part	Responsibility	CEMS example
Frontend / user interface	Presents information and collects user actions	Student event list and registration screen
Backend / application service	Applies business rules and coordinates operations	Checks event status, capacity, and duplicate registrations
Database	Stores persistent information	Students, events, registrations, and timestamps
Authentication / authorisation	Identifies users and controls permissions	Student versus organiser access
External services	Provides supporting capabilities	Email notifications or college login, if integrated

Why design before implementation?

Design makes important decisions visible before they become expensive to change. If the team discovers late that its data model cannot represent cancellations or waitlists, it may need to alter database structures, APIs, screens, and tests. Early design does not eliminate change, but it helps the team recognise dependencies and make informed choices.

Design artefacts

Depending on the project, design outputs can include wireframes, architecture diagrams, UML diagrams, database schemas, API specifications, security decisions, and a description of error handling. Not every project needs every document. The goal is to provide enough shared understanding for the team to build and review the solution.

> **DESIGN OUTPUT**
> A practical blueprint for the system: architecture, user-interface sketches, data structures, interfaces, permissions, and important technical decisions.

6. Phase Four — Implementation

Implementation is where developers turn the design into working software. They write code, configure services, create database operations, connect interfaces, and integrate components. Teams often use source control so changes can be reviewed, tracked, and combined safely.

For CEMS, implementation might be split into features: event listing, event creation, registration, cancellation, and reporting. One developer may work on the frontend, another on backend APIs, and another on database integration. Shared definitions and interface agreements help prevent the components from disagreeing about data formats or behaviour.

Good implementation practices

- Use clear names and consistent coding conventions.
- Keep related responsibilities organised into understandable modules.
- Validate user input and enforce important rules on the server, not only in the browser.
- Use version control and meaningful commits.
- Review code and run automated checks where practical.
- Avoid exposing secrets or private data in source code and logs.
- Record important decisions so future developers can understand them.

Why does implementation not end the work?

Code that compiles may still have incorrect business rules, weak security, poor usability, performance problems, or integration defects. The next phase—testing—provides structured evidence about whether the product behaves as expected. In many teams, developers also test their own code during implementation.

CEMS example: duplicate registration

Suppose a student double-clicks the Register button. The frontend may disable the button after the first click, but a reliable system should also enforce the rule in backend logic and the database. Otherwise, two requests could arrive quickly and create duplicate registrations. This is an example of a design rule that must be implemented consistently across relevant layers.

Implementation output

The main output is working software: source code, configuration, database scripts, automated checks, and the integrated components that can be tested. The code should be versioned so the team can trace changes and return to a known version if needed.

7. Phase Five — Testing

Testing checks whether the software behaves as expected and helps uncover defects. Testing should be connected to requirements: if a requirement matters, the team should be able to explain how it will be verified. Testing is not only a final activity; reviews and automated checks can happen throughout development.

A simple CEMS test

Test element	Example
Precondition	An event is published and has one seat remaining.
Action	Two different students attempt to register at nearly the same time.
Expected result	The system accepts no more than one additional registration and clearly informs the other student.
Evidence	Registration records, response messages, and test results are captured.

What kinds of situations should be tested?

- Normal use: a student registers for an event with available seats.
- Invalid input: required fields are missing or a malformed value is submitted.
- Boundary conditions: the event is full or exactly one seat remains.
- Permissions: a student attempts to access organiser-only functions.
- Integration: confirmation is sent only when the relevant registration result is valid.
- Failure recovery: a network timeout occurs during a request.
- Regression: an update to cancellation does not break registration or reporting.
- Performance: the system is evaluated under an agreed workload.

Testing terms to know

Term	Meaning	CEMS example
Test case	A set of conditions, steps, and expected results	Attempt duplicate registration
Defect / bug	A behaviour that differs from expected requirements	A full event accepts another registration
Regression testing	Rechecking existing behaviour after a change	QR attendance update does not break registration
Verification	Checking whether work products meet specified requirements	Review the design against documented rules
Validation	Checking whether the solution meets real user needs	Ask organisers to try the workflow and provide feedback

Can testing prove there are no bugs?

Usually not. A system can have too many possible inputs, states, users, devices, and failure conditions to test every combination. Testing reduces uncertainty and provides evidence. Good teams prioritise tests based on risk, usage, complexity, and consequences of failure. They also monitor the live system so they can detect issues that escaped testing.

> **TESTING OUTPUT**
> Test cases, test data, test execution results, defect reports, regression results, and a summary of remaining risks or limitations.

8. Phase Six — Deployment

Deployment makes a software release available in its intended environment. This step can involve more than copying files to a server. The team may need to configure environments, set permissions, migrate data, establish backups, enable monitoring, train users, and communicate the release.

For CEMS, releasing directly to every student without checking readiness could create avoidable disruption. A safer rollout might begin with a test environment, then a small group of organisers, and finally the wider student population. The exact strategy depends on the risk and constraints.

Deployment readiness checklist

- Have the important tests passed, and are known defects understood?
- Are production settings and access permissions correct?
- Is important data backed up, and is any migration checked?
- Do users know how to access the system and where to get help?
- Can the team monitor errors and performance after release?
- Is there a recovery or rollback plan if a serious problem occurs?

Deployment versus release

These terms are sometimes used differently across organisations. In general, deployment refers to installing or making a version available in an environment. Release refers to making a capability available to users. A team may deploy a feature but keep it hidden until it is ready to be released.

CEMS example

The team deploys the portal to production, checks its health, and enables access for organisers. After the organisers confirm that event creation and reports work, the college announces registration to students. The team watches error rates and registration counts during the launch period.

Deployment output

The output is an accessible release plus the operational information needed to run it: configuration, release notes, support instructions, monitoring, and recovery arrangements.

9. Phase Seven — Maintenance

After release, real users interact with the system in ways that may not have been fully anticipated. Defects are reported, security vulnerabilities are discovered, browsers and services change, and stakeholders request new features. Maintenance keeps the product useful, secure, and dependable.

Common maintenance categories

Category	Meaning	CEMS example
Corrective	Fixing defects	Fix a bug that prevents a student from cancelling within the allowed period
Adaptive	Adjusting to a changed environment	Update integration after the college changes its login service
Perfective	Improving or extending capability	Add waitlists or improve organiser reports
Preventive	Reducing future problems	Refactor fragile code or update an outdated dependency

Maintenance is not an afterthought

A release creates ongoing responsibilities. Someone must investigate incidents, prioritise fixes, protect data, update dependencies, and decide which feature requests are worth implementing. A feature that sounds useful may also introduce security risks, support costs, and more testing. Maintenance decisions should therefore use the same engineering discipline as the initial build.

Feedback closes the loop

After the first college festival, organisers may ask for QR attendance, certificates, and feedback surveys. The team should understand the need, estimate the impact, prioritise it, update requirements and design, implement the chosen change, test it, and deploy it carefully. In this way, the lifecycle begins another cycle of improvement.

> **MAINTENANCE OUTPUT**
> A better-supported product, with defect fixes, updates, operational knowledge, user feedback, and a prioritised list of future improvements.

10. One Request, Seven Phases

Let us trace one feature through the complete SDLC: students should be able to cancel event registrations before a deadline.

Phase	What the team does for cancellation
Planning	Decides whether cancellation belongs in the first release and identifies the policy owner.
Requirements	Clarifies who may cancel, the deadline, confirmation behaviour, and whether a cancelled seat becomes available again.
Design	Defines the cancellation flow, status changes, database rules, and how reports show cancellations.
Implementation	Builds the cancellation action, server-side checks, and the related data updates.
Testing	Checks valid cancellation, expired deadlines, repeated clicks, permissions, and whether capacity is updated correctly.
Deployment	Releases the feature with correct settings and communicates the cancellation rule to users.
Maintenance	Fixes reported issues and adapts the rule if college policy changes.

What if the requirement changes?

Suppose the college changes the policy: cancellations are no longer allowed after an event begins. The team should not simply change one screen label. It must identify all affected parts: requirement documents, backend rules, user messages, reports, tests, and possibly support instructions. After the change, regression tests should check that ordinary registration and other cancellation scenarios still work.

A practical change-impact sequence

1. Clarify the request and the reason behind it.
2. Identify the affected requirements, screens, services, data, reports, and users.
3. Estimate the development, testing, deployment, and support impact.
4. Decide whether to accept, defer, simplify, or reject the change.
5. Update shared requirements, tasks, design decisions, and test cases.
6. Implement, review, test, and release the agreed change.

> **LESSON**
> A lifecycle gives the team a repeatable way to manage change. It does not mean that change is forbidden; it means changes should be understood, prioritised, and verified.

11. Classroom Activity — Map the SDLC

Work in groups. Your college wants to launch the CEMS portal in time for its annual technical festival. Use the scenario below to identify work in each SDLC phase.

Scenario

The festival has 12 events. Each event has a capacity limit. Students must sign in before registering. Organisers need registration reports, and the college wants the portal to work on mobile phones. Registration opens in three weeks. The team has limited time and cannot deliver every possible feature at launch.

Group task

1. Write the problem statement in two or three sentences.
2. Identify at least four stakeholders and one need for each.
3. List four functional and two non-functional requirements.
4. Choose three design decisions the team must make.
5. Write three test cases, including one boundary or failure case.
6. List two deployment-readiness checks.
7. Suggest two features that could be considered for a later release and explain why.

Discussion prompts

- What is the minimum useful version the college should release first?
- Which risk could most seriously affect the festival?
- How would you respond if a new payment requirement appeared one week before launch?
- What evidence would convince you that the portal is ready for students?

There may be several reasonable solutions. The quality of your answer depends on how clearly you explain assumptions, priorities, risks, and trade-offs.

12. Knowledge Check

Answer the questions before checking the suggested answers.

1. What is the purpose of the SDLC?

2. What is the difference between a functional and a non-functional requirement?

3. Name three feasibility areas a team may consider.

4. Why should requirements be testable?

5. Give two examples of design decisions for CEMS.

6. Why is checking duplicate registration only in the frontend insufficient?

7. What is regression testing, and when might it be needed?

8. What should a deployment recovery plan help the team do?

9. Name two kinds of maintenance and give an example of each.

10. Does the SDLC mean a team must never revisit an earlier phase? Explain.

13. Suggested Answers

1. The SDLC organises the work of planning, defining requirements, designing, building, testing, deploying, and maintaining software. It helps manage quality, cost, schedule, risks, and change.

2. A functional requirement states a service or behaviour, such as registering for an event. A non-functional requirement states a quality or constraint, such as limiting access to private reports or meeting an agreed response-time target.

3. Technical, economic, operational, schedule, and legal or policy feasibility are examples.

4. Testable requirements give the team a clear way to check whether the expected behaviour or quality has been achieved. Vague requirements create disagreements and make acceptance difficult.

5. Examples include defining the database structure for events and registrations, separating student and organiser permissions, designing the registration flow, or deciding how event capacity is enforced.

6. Requests can bypass the frontend, be repeated, or arrive concurrently. Important rules must also be enforced by trusted backend logic and, where appropriate, database constraints.

7. Regression testing rechecks existing behaviour after a change. For example, after adding cancellation or QR attendance, the team retests registration, capacity, and organiser reports.

8. It should help the team restore a known working version or otherwise recover safely if the release causes a serious problem.

9. Corrective: fix a cancellation bug. Adaptive: update the portal when the college login service changes. Perfective: improve reports. Preventive: update dependencies or refactor risky code.

10. No. The lifecycle is a way to organise work, and teams may revisit requirements, design, implementation, or testing when new information or defects arise.

14. Glossary

Term	Meaning
SDLC	Software Development Life Cycle; a structured view of software work from planning through maintenance.
Feasibility	An assessment of whether a proposed project is practical under its constraints.
Scope	The boundaries of what the project or release includes and excludes.
Requirement elicitation	Discovering user needs and constraints through interviews, observation, workshops, and other methods.
Acceptance criteria	Conditions used to decide whether a requirement or feature is acceptable.
Architecture	The high-level organisation of a system and the relationships among its major parts.
Implementation	Building the software and integrating its components.
Regression testing	Rechecking existing functionality after a change.
Deployment	Making a software version available in an intended environment.
Maintenance	Correcting, adapting, improving, and supporting software after release.

15. Homework — Apply the SDLC to an App You Use

Choose one digital service you know well, such as a UPI app, food delivery app, online shopping platform, college portal, or navigation app. Write a one- to two-page explanation using the prompts below.

1. Describe the problem the app solves and its main users.
2. Write three functional requirements and two non-functional requirements.
3. Explain one planning decision the team would need to make.
4. Describe one design decision and why it matters.
5. Write one normal test and one failure or boundary test.
6. Explain what a safe deployment might involve.
7. Suggest one maintenance improvement based on a realistic user need.

> **EXAMPLE**
> For a food delivery app, one functional requirement could be: “A customer can cancel an order while it is still in an eligible status.” A useful test would check cancellation at the permitted boundary and verify that the user sees the correct outcome.

Before the next class

Review the difference between the SDLC and a process model. In the next lesson, you will explore how Waterfall and Incremental development organise the lifecycle differently. Bring an example of a project where requirements are likely to remain stable and another where they may change frequently.

Final thought

A reliable software product is the result of connected decisions, not a single coding session. Planning gives the team direction, requirements define what matters, design makes the solution understandable, implementation builds it, testing checks it, deployment introduces it to users, and maintenance keeps it useful. The SDLC helps teams manage those responsibilities as one connected journey.

End of Day 2 • The Software Development Life Cycle